

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Цветовые модели». Вариант 4: RGB ↔ HSV ↔ LAB

Выполнил(а)	Никончук Андрей
Подгруппа	11Б

1. Цель и постановка задачи

Цель работы — изучить взаимные преобразования цветовых моделей и разработать веб-приложение, в котором один цвет одновременно представлен в RGB, HSV и CIELAB. Для варианта 4 реализованы RGB ↔ HSV ↔ LAB. Пользователь может задавать компоненты числовыми значениями, ползунками и через палитру; после изменения любой координаты остальные представления автоматически пересчитываются. В числовых полях и на ползунках ввод и отображение организованы только целыми числами, при этом истинные вычисленные координаты текущего цвета внутри программы сохраняются без округления.

2. Средства разработки и архитектура

Приложение выполнено на HTML5, CSS3 и чистом JavaScript (ES modules), без библиотек цветовой конвертации и без фреймворков. Архитектура разделена на Model–View–Controller: математический слой не работает с DOM, View отвечает за элементы страницы, Controller связывает пользовательские события с расчетами.

Слой	Назначение
Model	hsv.js, rgbXyz.js, lab.js, illuminants.js, matrix.js, converter.js — формулы, матрицы и gamut mapping.
View	appView.js, gradientSlider.js — поля, палитры, ползунки, вывод и динамические градиенты.
Controller	colorController.js — выбор маршрута преобразования и синхронизация трех моделей.

3. Алгоритмы преобразования

RGB ↔ HSV реализовано вручную через C_{max} , C_{min} и Δ . Для RGB ↔ LAB используется цепочка RGB ↔ XYZ ↔ LAB. Перед RGB → XYZ выполняется обратная gamma-коррекция sRGB; после XYZ → RGB — прямая gamma-коррекция. Преобразование XYZ ↔ LAB выполняется относительно текущей точки белого.

Готовые RGB ↔ XYZ-матрицы в рабочем коде отсутствуют. Хранятся только ху-координаты первичных цветов sRGB и источников D65, D50, E. Из primaries строится P, из ху источника вычисляется W, затем $S=P^{-1}W$ и $M=P \cdot \text{diag}(S)$. Обратная M^{-1} вычисляется собственной функцией `inverse3()`. Поэтому при смене источника обе матрицы пересчитываются во время выполнения.

При LAB → RGB возможен выход линейных каналов за $[0;1]$. Реализованы две стратегии: Clipping независимо ограничивает каждый канал, Scaling совместно масштабирует диапазон компонентов. При выходе за gamut интерфейс выводит предупреждение и исходные линейные RGB до коррекции.

4. Интерфейс и дополнительные требования

На странице одновременно отображаются RGB, HSV и LAB. Для всех девяти компонент есть числовое поле и ползунок; в каждой панели доступна системная палитра. Пользовательские поля и положения ползунков используют целочисленный шаг 1 и отображают только целые значения. Округление выполняется только в слое View: Controller хранит исходные вычисленные RGB/HSV/LAB с полной точностью и при следующем изменении берет неизменяемые координаты из точного результата, а не из округленных полей. Фоны ползунков динамические: при изменении любого компонента вычисляются новые цвета с учетом точных текущих остальных координат. Для LAB образцы градиента проходят LAB → XYZ → RGB с выбранными D65/D50/E и Clipping/Scaling.

В диагностическом блоке отображаются текущая точка белого, промежуточные XYZ и рассчитанная RGB → XYZ-матрица. Это позволяет на защите визуально показать, что при D65 → D50 → E матрица действительно меняется.

5. Автоматическое тестирование

Математический слой и конфигурация интерфейса покрыты 29 микро-тестами. Они запускаются прямо в браузере по адресу `/tests/` и импортируют те же модули, что и основное приложение; Node.js и внешняя тестовая библиотека не требуются. Проверяются RGB ↔ HSV, RGB ↔ XYZ для D65/D50/E, XYZ ↔ LAB, матричная математика, динамическая перестройка матрицы, Clipping/Scaling, выход за gamut, ColorConverter, а также целочисленный шаг и отсутствие дробного отображения компонентов. Контрольный пример RGB(255,0,0) дает HSV(0°,100%,100%) и LAB приблизительно (53,24; 80,09; 67,20) при D65; во View LAB будет показан как целые 53, 80, 67, однако внутренние значения остаются дробными.

6. Запуск и публикация

Локальный запуск выполняется командой «`python -m http.server 8000`» из корня проекта; Python используется только как статический HTTP-сервер, а все расчеты выполняются JavaScript в браузере. Приложение открывается по `http://localhost:8000/`, тесты — по `http://localhost:8000/tests/`.

Для сдачи проект подготовлен к GitHub Pages: сборка не требуется, пути к ресурсам относительные, добавлен .nojekyll. После загрузки исходников в public-репозиторий необходимо включить Settings → Pages → Deploy from a branch → main → /(root). Преподаватель получает публичную ссылку на приложение и отдельную ссылку `/tests/`.

7. Соответствие требованиям и вывод

Выполнены базовые требования: три модели одновременно, три способа задания цвета, автоматический пересчет, целочисленный пользовательский ввод/отображение без потери точности внутренних координат и предупреждение о некорректном RGB. Выполнены дополнительные требования подгруппы 10А: ручная математика, D65/D50/E с динамическими матрицами, Clipping/Scaling, динамические градиенты, отдельные Model/View/Controller и автоматические микро-тесты. UCR/GCR не применяются, поскольку они относятся к расчету CMYK, а в варианте 4 CMYK отсутствует.

В результате разработано автономное статическое веб-приложение, пригодное для локального запуска и публикации на GitHub Pages без серверной части и сторонних JavaScript-зависимостей.

Исходные материалы: «Лабораторная работа 1»; «Формулы преобразования цветовых моделей».